ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Alliance School of Advanced Computing

Course Name: Graph Analytics
Course Code: E1ITC 302
Assignment No: 3

Name of the Student: G. Sowmya

Registration Number: 2411021060512

A. Title of The Problem:

Kruskal (MST): Really Special Subtree

B. Problem Statement in details:

Given an undirected, weighted, connected graph with *n* nodes numbered consecutively from 1 to *n* and *m* edges, the task is to find a **Really Special SubTree (RSS)**: a subtree that spans every node of the graph, contains no cycles, and has the minimum possible total edge weight. In other words, the problem asks for the Minimum Spanning Tree (MST) of the graph and the sum of the weights of its edges.

Each edge is described by three integers: two endpoint nodes *u* and *v*, and a weight *wt*. Multiple edges may share the same weight. When several edges have equal weight, they must be ordered by the sum *u* + *v* + *wt* (smaller sum first); if there is still a tie, any of the tied edges may be chosen. The function `kruskals(g_nodes, g_from, g_to, g_weight)` must return a single integer: the total weight of the Really Special SubTree.

Input format: The first line contains two space-separated integers, *g_nodes* and *g_edges*. Each of the next *g_edges* lines contains three space-separated integers *g_from*, *g_to* and *g_weight*.

Constraints: $2 \le n \le 3000$; $1 \le m \le n(n-1)/2$; $1 \le u, v \le n$; $0 \le wt \le 10^5$. The graph is guaranteed to be connected.

Output format: A single integer denoting the total weight of the Really Special SubTree.

Sample Input: 4 nodes and 6 edges: (1,2,5), (1,3,3), (4,1,6), (2,4,7), (3,2,4), (3,4,5). **Sample Output:** 12 — the MST uses edges (1,3,3), (3,2,4) and (3,4,5), giving 3 + 4 + 5 = 12.

C. HackerRank/Leetcode Link of the problem (If applicable):

https://www.hackerrank.com/challenges/kruskalmstrsub/problem

D. Student's Github link of report and code upload, along with the readme file:

https://github.com/iamsowmya104/Assignment-2

E. Solution Steps/Algorithm:

1. Read the number of nodes and edges, then read every edge (*u*, *v*, *weight*) and store it as an `Edge` object in a list.
2. Sort all edges in non-decreasing order of weight. If two edges have the same weight, break the tie using the sum *u* + *v* + *weight*, as required by the problem. This is implemented by making `Edge` implement `Comparable`.
3. Create a Disjoint Set Union (DSU / Union-Find) structure of size *n* + 1 in which every node is initially its own parent. Use path compression in `find()` to keep the trees shallow.
4. Initialise `totalWeight = 0` and `edgesCount = 0`.
5. Traverse the sorted edge list. For each edge, find the roots of its two endpoints.

6. If the roots are different, the edge connects two separate components and does not form a cycle: union the two sets, add the edge weight to `totalWeight` and increment `edgesCount`. If the roots are the same, the edge would create a cycle, so skip it.
7. Stop as soon as `edgesCount == n – 1`, since a spanning tree of *n* nodes has exactly *n* – 1 edges. Return `totalWeight` as the weight of the Really Special SubTree.

F. Code developed:

```
import java.io.*;
import java.util.*;

class Edge implements Comparable<Edge> {
    int u, v, weight;

    public Edge(int u, int v, int weight) {
        this.u = u;
        this.v = v;
        this.weight = weight;
    }

    @Override
    public int compareTo(Edge other) {
        if (this.weight != other.weight) {
            return Integer.compare(this.weight, other.weight);
        }
        int sumThis = this.u + this.v + this.weight;
        int sumOther = other.u + other.v + other.weight;
        return Integer.compare(sumThis, sumOther);
    }
}

class DisjointSet {
    int[] parent;

    public DisjointSet(int n) {
        parent = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            parent[i] = i;
        }
    }

    public int find(int i) {
        if (parent[i] == i)
            return i;
        return parent[i] = find(parent[i]);
    }

    public boolean union(int i, int j) {
        int rootI = find(i);
        int rootJ = find(j);
        if (rootI != rootJ) {
            parent[rootI] = rootJ;
            return true;
        }
        return false;
    }
}

class Result {

    public static int kruskals(int gNodes, List<Integer> gFrom, List<Integer> gTo, List<Integer> gWeight) {
        List<Edge> edges = new ArrayList<Edge>();
        int m = gFrom.size();

        for (int i = 0; i < m; i++) {
```

```
            edges.add(new Edge(gFrom.get(i), gTo.get(i), gWeight.get(i)));
        }

        Collections.sort(edges);

        DisjointSet dsu = new DisjointSet(gNodes);
        int totalWeight = 0;
        int edgesCount = 0;

        for (Edge edge : edges) {
            if (dsu.union(edge.u, edge.v)) {
                totalWeight += edge.weight;
                edgesCount++;
                if (edgesCount == gNodes - 1) {
                    break;
                }
            }
        }

        return totalWeight;
    }
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        String[] gNodesEdges = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int gNodes = Integer.parseInt(gNodesEdges[0]);
        int gEdges = Integer.parseInt(gNodesEdges[1]);

        List<Integer> gFrom = new ArrayList<Integer>();
        List<Integer> gTo = new ArrayList<Integer>();
        List<Integer> gWeight = new ArrayList<Integer>();

        for (int i = 0; i < gEdges; i++) {
            String[] gFromToWeight = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

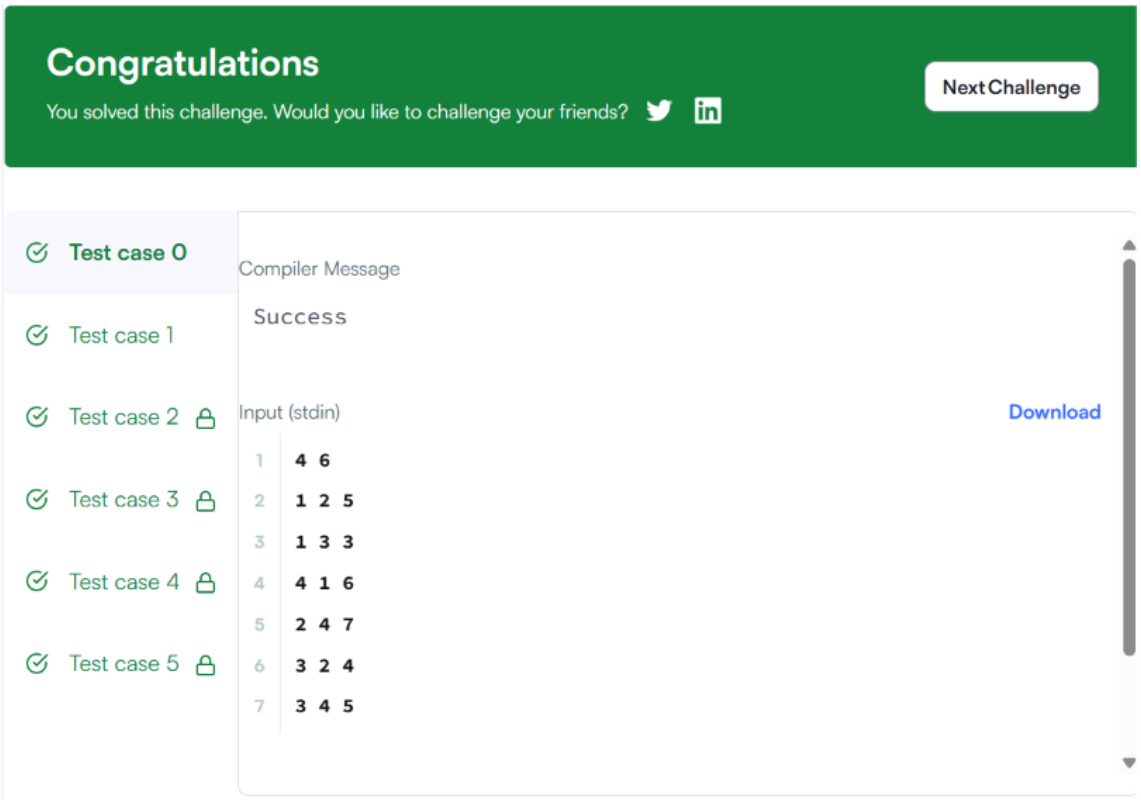
            gFrom.add(Integer.parseInt(gFromToWeight[0]));
            gTo.add(Integer.parseInt(gFromToWeight[1]));
            gWeight.add(Integer.parseInt(gFromToWeight[2]));
        }

        int res = Result.kruskals(gNodes, gFrom, gTo, gWeight);

        bufferedWriter.write(String.valueOf(res));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

G. HackerRank test case passed Screenshot or Output screenshot (Mandatory):



H. Observation:

The program was implemented in Java using Kruskal's algorithm together with a Disjoint Set Union (Union-Find) data structure to compute the weight of the Minimum Spanning Tree of an undirected, weighted graph. All edges are sorted by weight, with ties resolved by the sum $u + v +$ *weight* as the problem requires, and an edge is accepted only when its endpoints belong to different components, which guarantees that no cycle is formed. Path compression in the `find()` operation keeps the DSU operations close to constant time, so the overall time complexity is dominated by sorting, i.e. $O(E \log E)$, with $O(V + E)$ space. The loop terminates early once $n - 1$ edges have been selected, avoiding unnecessary iterations over the remaining edges. The implementation produced the correct total weight for the sample case (12) and passed all six test cases (Test case 0 to 5) on HackerRank, including the hidden ones, within the time and memory limits.